Queue

So far, we have studied the queue abstract data structure and its interface. We have also learnt how to implement the queue using a linked list.

Keeping the interface same, can we implement the queue using an array?

Let us examine the design.

Interface of the queue:

Apart from initialization and deinitialization, the queue supports the following operations.

initialize()	Create an empty queue
enqueue(x)	Insert x at the rear
dequeue()	Remove the element at the front
front() / peek()	Examine the front element
isEmpty()	Check whether the queue is empty
isFull()	Check whether the queue is full, for bounded implementations
destroy() / deinitialize()	Release resources

Implementation : We are implementing the queue using an array. A queue has to have two ends the front and the rear. The queue would contain an array of components and two indices the front and the rear.

```
#define MAXSIZE 5

struct queue

{

        int key_[MAXSIZE];

        int front_;

        int rear_;

};

typedef struct queue queue_t;
```

For illustration purpose, the component type is chosen as an int.

Design consideration : How to handle the indices front and rear in enqueue and dequeue?

a) Incrementing indices correctly

Initially both these could be same – some valid index. Could be zero. On enqueue we should increment the rear and deque we should increment the front.

The valid indices can be anything in between 0 and MAXSIZE – 1. If we keep incrementing, the index can go outside the array boundary. Or if we were to say that the queue is full when the rear reaches MAXSIZE, it could be wrong if there have been a few deque operations.

To avoid the index going outside the array bounds and to avoid saying that the queue is full when some elements have been freed by deque, we will use **modulo arithmetic**.

To increment the index say i, we shall say

i = (i + 1) % MAXSIZE

b) Checking for queue empty and queue full

If the rear catches the front, queue is full.
If the front catches the rear, queue is empty.
These could be fairly complicated conditions.
Can we make that simple by using the same condition for both queue full and queue empty?
We can provided we waste one location.
But this might be checked at different points in time enqueue and dequeue.
We shall discuss at length in these operations.

Implementation:

- **Initialize the list**

  Both the front and the rear of the queue are initialized to any valid index.

  We are initializing it to 0.

```
void init_queue(queue_t *ptr_queue)

{

    // could be any valid index in the array

    ptr_queue->rear_ = ptr_queue->front_ = 0;

}
```

- **Deinitalize the list**

  As there are no resources allocated while adding to the queue, resetting the rear and the front to any valid index will be sufficient.

```
void deinit_queue(queue_t *ptr_queue)

{

    ptr_queue->rear_ = ptr_queue->front_ = 0;

}
```

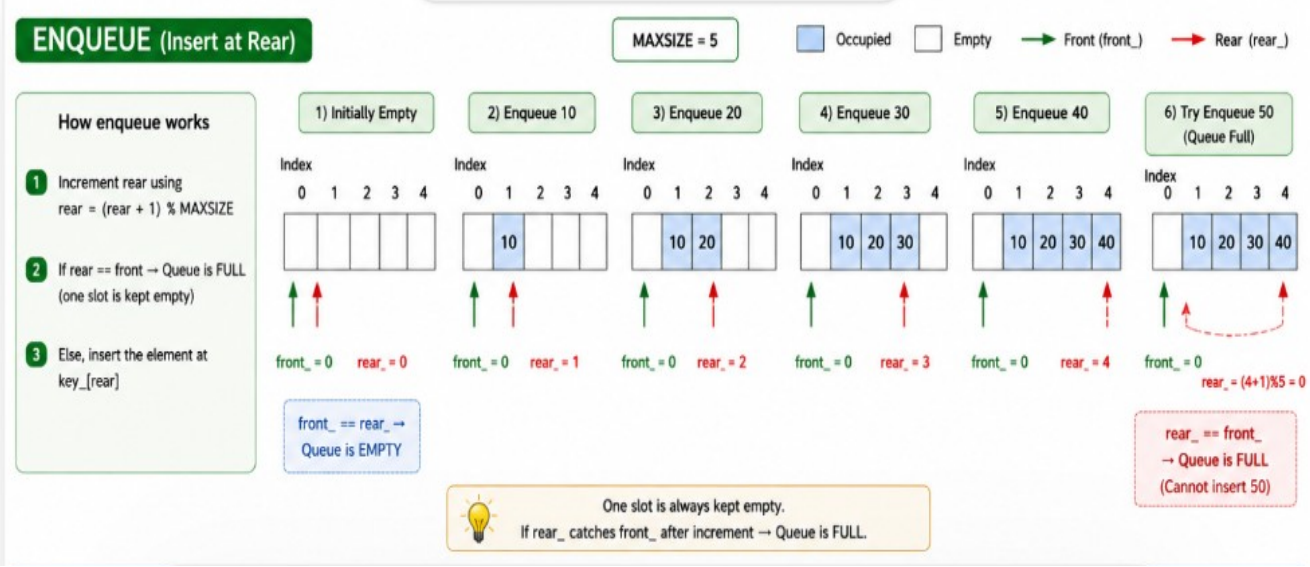
- **Enqueue**

  If the rear on incrementing using modular arithmetic catches the front, the queue is full.

  Note that we first increment the rear and compare with the front for checking for queue being full.

  If the queue is not full, use rear as the index and assign the component to the array.

  If the queue is full, reverse the change to rear.




```
void enque(queue_t *ptr_queue, int key)
{
    int temp;
    temp = ++ptr_queue->rear_ % MAXSIZE;
    ptr_queue->rear_ = temp;
    if(! is_full(ptr_queue))
    {
            ptr_queue->key_[temp] = key;
    }
    else
    {
                ptr_queue->rear_ = (ptr_queue->rear_ - 1 + MAXSIZE) % MAXSIZE;
                printf("queue full; cannot push\n");
    }
}
```

Should we take care of queue being empty as a special case?

==========

- **Dequeue** :

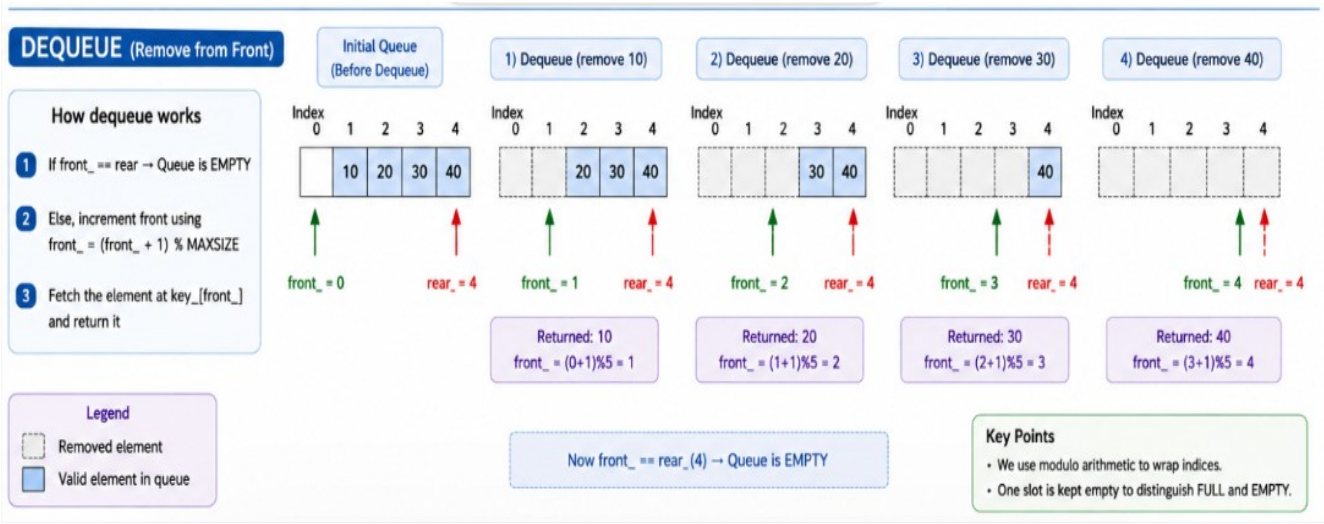
  We cannot deque if queue is empty. Remember the initialization of the queue. Both the front and the rear had the same value.

  We are wasting one location to make the condition checking for both queue full and queue empty same.

  If the front and the rear are equal before any incrementing, queue is empty.

If the front and the rear are equal after incrementing rear, queue is full.

If the queue is not empty, increment front using modulo arithmetic, use that the index and fetch the component.




```
int deque(queue_t *ptr_queue)
{
    if(! is_empty(ptr_queue))
    {
            int key;  int temp = ++ptr_queue->front_ % MAXSIZE;
            key = ptr_queue->key_[temp];
            ptr_queue->front_ = temp;
            return key;
    }
    else
    {
            printf("queue empty; cannot pop\n");
            return '\0';
    }
}
```

- **is_empty & is_full**

The condition is same in both – but at what time it is called in enqueue and deque differs.
To check queue is empty, deque calls this function before incrementing front.
To check queue is full, enque calls this function after incrementing rear.

```
// to be called before deque before incrementing

int is_empty(queue_t *ptr_queue)

{

    return ptr_queue->front_ == ptr_queue->rear_;

}


// to be called in enque after incrementing

int is_full(queue_t *ptr_queue)

{

    return ptr_queue->front_ == ptr_queue->rear_;

}
```

As expected, and required, the client code remains unaffected.